

Module 3 Exercises

Databases, Schemas & Tables

Environment: Yellowbrick Cloud Test Environment

Shared sample data: `samples.retail` — retail schema accessible to all students

Tools: Yellowbrick Manager (SQL Editor + GUI Object Browser)

Part	What You Will Do
1	Explore the environment — databases, schemas, and tables via GUI and SQL
2	Create your personal database and schema
3	Design and CREATE a table with proper data types, distribution, and ordering
4	Populate your table with data from the retail sample database (INSERT from SELECT)
5	Work with temporary tables — create, query, observe them disappear on disconnect
Stretch	Inspect your table's storage metadata using sys views

✓ **Note**

Some of these exercises use SQL that we haven't learned yet. Tomorrow's class will go over some of those SQL Statements in more detail. Just follow along for now.

Part 1 — Explore the Environment

Before creating anything, get familiar with what already exists — both through the Yellowbrick Manager GUI and with SQL.

Time: ~15 minutes

1.1 Use the GUI to explore databases and schemas

Yellowbrick Manager

In Yellowbrick Manager, use the Object Browser panel on the left side:

1. Expand the Databases node to see all databases available to you.
2. Expand the samples database → retail schema → Tables to see the retail tables.
3. Click on any table name to see its columns, data types, and properties in the details panel.

This is the fastest way to navigate objects — use it alongside SQL.

1.2 What databases can you see?

Run this in the SQL Editor to list all databases on the system:

```
SELECT name AS database_name
       , encoding
FROM   sys.database
ORDER BY name;
```

Expected: You should see `samples` (the shared sample data), and possibly others.

1.3 What schemas are in your database?

Switch to the “Samples” database in the gui, then run

```
SELECT schema_name
       , schema_owner
FROM   information_schema.schemata
ORDER BY schema_name;
```

Note: `information_schema` only shows schemas in the database you are currently connected to. You should see `public` and system schemas.

1.4 Explore the retail sample data

The retail sample data is in a separate database called `samples`. You can query it directly using the three-part `database.schema.table` notation:

```
-- Preview the fact_sales table
SELECT *
FROM   samples.retail.fact_sales
LIMIT 5;

-- How many rows does it have?
SELECT COUNT(*) AS row_count
FROM   samples.retail.fact_sales;

-- What stores exist?
SELECT store_key, store_name, city, state
FROM   samples.retail.dim_store
ORDER  BY store_key
LIMIT 10;
```

Question: How many rows are in `fact_sales`? What columns does it have? Note the column names and data types — you will use this table in Part 4.

1.5 Confirm your current context

```
SELECT current_database() AS my_database
      , current_schema()  AS default_schema
      , current_user      AS logged_in_as;
```

Expected: `my_database = db_<username>` `default_schema = public` `logged_in_as = your username`

1.6 Exploration

The examples above used fully qualified sql queries to reference the database and schema. Using the GUI, change the database and schema to `Samples` and `Retail`, then confirm the context as above in 1.5. Then re-run the sql statements in 1.4, but with just the table names.

Part 2 — Create Your Own Database / Schema

Every analyst should have their own schema inside your assigned database as a private workspace. This is where all your tables will live during training and in future sessions.

Time: ~5 minutes

2.1 Create your Database and Schema

Replace `your_username` with your actual Yellowbrick username throughout all exercises:

```
-- Create your personal schema
CREATE DATABASE db_[username] with encoding=utf8;

-- Create your personal schema
CREATE SCHEMA exercises;

-- Verify it was created
SELECT schema_name, schema_owner
FROM   information_schema.schemata
WHERE  schema_name = 'exercises';
```



Yellowbrick Manager

After running `CREATE SCHEMA`, right-click your database in the Object Browser and choose Refresh. Your new schema will appear under Schemas.

2.2 Set your search_path

Setting your `search_path` means Yellowbrick will look in your schema first when you reference a table name without a prefix. This also ensures objects you create land in your schema, not `public`.

```
-- Set search_path for this session
SET search_path = exercises, public;

-- Verify it took effect
SHOW search_path;
```

⚠ Warning

`SET search_path` applies only to your current session. If you disconnect and reconnect, you will need to run it again.

In the GUI, every run is a new session. So every run, the path is reset to the path selected in the GUI. If you want to make your `search_path` permanent, use `ALTER USER` as below

```
ALTER USER your_username  
  SET search_path = exercises, public;
```

2.3 Confirm your schema is the default

```
SELECT current_database() AS my_database  
  , current_schema() AS default_schema  
  , current_user AS logged_in_as;
```

Expected: default_schema should now be exercises,public.

Part 3 — Design and Create a Table

You will create a summary table called `revenue_by_store_month` to hold aggregated sales revenue per store per month. Think carefully about each design decision — the choices you make affect query performance.

Time: ~15 minutes

3.1 Review the design decisions

Before writing the DDL, think through each column:

Column	Data Type	Why this type?
<code>store_key</code>	INTEGER NOT NULL	Foreign key to <code>dim_store</code> — whole number, never null
<code>revenue_month</code>	DATE NOT NULL	First day of each month — no time component needed
<code>total_revenue</code>	NUMERIC(18,2) NOT NULL	Money — exact decimal, up to 18 digits, 2 decimal places
<code>transaction_count</code>	INTEGER NOT NULL	Count of transactions — whole number, never null
<code>avg_unit_price</code>	NUMERIC(12,2)	Average price — exact decimal, can be NULL if no sales
<code>loaded_at</code>	TIMESTAMP	When this row was loaded — date + time, nullable

Distribution key: We will join this table back to `dim_store` on `store_key`. Distributing on `store_key` means those joins can happen locally on each compute node — no network shuffle.

Sort key: Most reporting queries will filter by date range (e.g. `WHERE revenue_month BETWEEN ...`). Sorting on `revenue_month` lets the engine skip entire blocks of data outside the date range — this is block pruning.

3.2 Write the CREATE TABLE statement

Try writing it yourself based on the table above, then check against the working solution below:

```
CREATE TABLE exercises.revenue_by_store_month
(
  store_key      INTEGER      NOT NULL,
  revenue_month  DATE         NOT NULL,
  total_revenue  NUMERIC(18,2) NOT NULL,
  transaction_count INTEGER    NOT NULL,
  avg_unit_price NUMERIC(12,2),
  loaded_at      TIMESTAMP
)
DISTRIBUTE ON (store_key)
SORT ON (revenue_month);
```

**Tip**

SORT ON takes a single column — this is the primary ordering key for block pruning. Choose the column you filter on most in **WHERE** clauses, typically a date.

3.3 Verify the table was created correctly

```
-- Check columns and data types
SELECT column_name, data_type, is_nullable
FROM   information_schema.columns
WHERE  table_schema = 'exercises'
AND    table_name   = 'revenue_by_store_month'
ORDER  BY ordinal_position;

-- Check distribution and sort key
SELECT name, distribution, distribution_key, sort_key
FROM   sys.table
WHERE  name = 'revenue_by_store_month';
```

Questions: Are NOT NULL columns showing `is_nullable = 'NO'`? Does `sys.table` show `distribution = 'hash'` with `distribution_key = 'store_key'` and `sort_key = 'revenue_month'`?

**Yellowbrick Manager**

In Yellowbrick Manager, expand your schema in the Object Browser → Tables → `revenue_by_store_month`. Click the table to see its columns, types, and constraints in the detail panel — no SQL required.

Part 4 — Populate Your Table

Your table is empty. You will populate it by running an aggregation query against the retail sample data and inserting the results. This is the most common way to build reporting tables in Yellowbrick.

Time: ~10 minutes

4.1 Understand the source data

Look at the source table first. Note that `fact_sales` has a `transaction_timestamp` column (not a date) and a `total` column for the sale amount:

```
-- Preview a few rows from the source
SELECT store_key
       , transaction_timestamp
       , total
       , unit_price
FROM   samples.retail.fact_sales
LIMIT 5;

-- Check the row count
SELECT COUNT(*) AS total_rows
FROM   samples.retail.fact_sales;
```

4.2 Insert aggregated data into your table

Use `DATE_TRUNC('month', timestamp)::DATE` to group transactions by month. The `::DATE` cast converts the result to a DATE value:

```
INSERT INTO exercises.revenue_by_store_month
(store_key, revenue_month, total_revenue,
 transaction_count, avg_unit_price, loaded_at)
SELECT
 store_key,
 DATE_TRUNC('month', transaction_timestamp)::DATE,
 SUM(total),
 COUNT(*),
 ROUND(AVG(unit_price)::NUMERIC, 2),
 CURRENT_TIMESTAMP
FROM   samples.retail.fact_sales
GROUP BY store_key
       , DATE_TRUNC('month', transaction_timestamp)::DATE;
```


4.3 Verify the data loaded correctly

```
-- How many rows were created?
SELECT COUNT(*) AS row_count
FROM   exercises.revenue_by_store_month;

-- Preview the results
SELECT *
FROM   exercises.revenue_by_store_month
ORDER  BY store_key, revenue_month
LIMIT 10;

-- Which month had the highest total revenue across all stores?
SELECT revenue_TO_CHAR(revenue_month, 'YYYY-MM') AS revenue_month
      , SUM(total_revenue) AS monthly_revenue
FROM   exercises.revenue_by_store_month
GROUP  BY revenue_month
ORDER  BY monthly_revenue DESC
LIMIT 5;
```

Questions: How many rows did your summary table produce? How does that compare to the ~5.5 million rows in fact_sales? Why is the summary table much smaller?

Part 5 — Temporary Tables

Temp tables are scoped to your session and dropped automatically when you disconnect. They are useful for breaking complex queries into steps and staging intermediate results without cluttering shared schemas.

Time: ~10 minutes

5.1 Create a temp table from a query

Build a temp table holding the top 10 stores by total revenue:

```
CREATE TEMP TABLE top_stores AS
SELECT
    store_key,
    SUM(total_revenue)      AS total_revenue,
    SUM(transaction_count) AS total_transactions
FROM   exercises.revenue_by_store_month
GROUP BY store_key
ORDER BY total_revenue DESC
LIMIT 10;
```

5.2 Query your temp table

NOTE: In YB Manager Query Editor, every “Run” is a new session. Therefore, you need to create the temp table and run the query on it in one “Run”

```
-- Query it like any table
SELECT *
FROM   top_stores
ORDER BY total_revenue DESC;

-- Join it back to the monthly summary for a breakdown
SELECT r.store_key
      , r.revenue_month
      , r.total_revenue
FROM   exercises.revenue_by_store_month AS r
JOIN   top_stores                      AS t
      ON r.store_key = t.store_key
ORDER BY r.store_key, r.revenue_month;

-- Enrich with store names from the dimension table
SELECT s.store_name
      , s.city
      , t.total_revenue
      , t.total_transactions
FROM   top_stores                      AS t
JOIN   samples.retail.dim_store        AS s
      ON t.store_key = s.store_key
ORDER BY t.total_revenue DESC;
```

Stretch Goal — Inspect Table Metadata

If you finish early, explore these queries to see how Yellowbrick stores information about your tables.

S.1 Check distribution and sort key

```
SELECT name          AS table_name
       , distribution
       , distribution_key
       , sort_key
FROM   sys.table
WHERE  name = 'revenue_by_store_month';
```

Expected: distribution = 'hash', distribution_key = 'store_key', sort_key = 'revenue_month'

S.2 Confirm NOT NULL is enforced

Try inserting a row with a NULL value in a NOT NULL column:

```
-- This should fail with a NOT NULL violation
INSERT INTO exercises.revenue_by_store_month
    (store_key, revenue_month, total_revenue, transaction_count)
VALUES
    (NULL, '2024-01-01', 50000.00, 100);

-- Expected error:
-- null value in column "store_key" violates not-null constraint
```



Tip

NOT NULL is the only constraint Yellowbrick enforces on load. Try inserting a duplicate store_key + revenue_month row — it will succeed, because UNIQUE and PRIMARY KEY are informational only in Yellowbrick.

Quick Reference

Task	SQL
Current context	<code>SELECT current_database(), current_schema(), current_user;</code>
List databases	<code>SELECT name FROM sys.database ORDER BY name;</code>
List schemas	<code>SELECT schema_name FROM information_schema.schemata ORDER BY 1;</code>
List tables in schema	<code>SELECT table_name FROM information_schema.tables WHERE table_schema = 'name';</code>
Inspect columns	<code>SELECT column_name, data_type, is_nullable FROM information_schema.columns WHERE table_schema='s' AND table_name='t';</code>
Create schema	<code>CREATE SCHEMA schema_name;</code>
Set search_path (session)	<code>SET search_path = my_schema, public;</code>
Set search_path (permanent)	<code>ALTER USER username SET search_path = my_schema, public;</code>
Create table	<code>CREATE TABLE s.t (col type NOT NULL, ...) DISTRIBUTE ON (col) SORT ON (col);</code>
Populate from SELECT	<code>INSERT INTO s.t (cols) SELECT ... FROM source;</code>
Create temp table	<code>CREATE TEMP TABLE t AS SELECT ...;</code>
Check distribution/sort	<code>SELECT name, distribution, distribution_key, sort_key FROM sys.table WHERE name='t';</code>
Drop table safely	<code>DROP TABLE IF EXISTS s.t;</code>
Drop schema + contents	<code>DROP SCHEMA IF EXISTS s CASCADE;</code>
Cross-database query	<code>SELECT * FROM other_db.schema_name.table_name;</code>